



Abril de 2025

Desenvolvimento de Games

*Relatório sobre cinco minigames elaborados
em Java e Processing*

Apresentado para

Leopoldo Neto

Preparado pela equipe

Kaleido Void Studios

DESENVOLVIMENTO DE GAMES (5º PERÍODO)

Minigame 1: Jogo do Marciano

Descrição

O Jogo do Marciano é um divertido e desafiador jogo em que o jogador assume o papel de um terráqueo tentando desvendar os segredos numéricos escondidos por um marciano brincalhão. Seu principal objetivo é descobrir um número secreto entre 01 e 99 dentro de um número limitado de tentativas. Se o jogador acertar, ele poderá acessar o Desafio Final, uma fase ainda mais intrigante que testa a lógica e atenção do jogador.

Diferenciais e Funcionalidades Principais da nossa versão

1. Dois modos de desafio: jogo principal e o Desafio Final;
2. Sistema de recordes: os melhores jogadores têm seu nome registrado;
3. Feedback personalizado: dicas sobre o número (maior/menor) e um sistema de feedback visual com símbolos no Desafio Final;
3. Experiência imersiva: introdução narrativa e interações com o "universo" do jogo;
4. Código secreto com lógica similar ao Mastermind ou Termo

Como Jogar

1. Jogo Principal:
 - O jogo escolhe aleatoriamente um número entre 01 e 99.
 - Você tem até 8 tentativas para adivinhar o número.
 - Após cada tentativa, o jogo dirá se o número secreto é maior ou menor.
 - Se acertar, você pode registrar seu nome no ranking e acessar o Desafio Final.

```
Bem-vindo ao Jogo do Marciano!
Em uma galáxia distante, um Marciano travesso escondeu um número entre 01 e 99.
Sua missão é adivinhar esse número antes que ele fuja para outra dimensão!
Você terá 8 tentativas para provar sua habilidade.
Boa sorte, Terráqueo!

-----

Novo jogo! Adivinhe o número entre 01 e 99.
Tentativas restantes: 7
Tentativa 1: Digite sua tentativa (01 a 99, ou 0 para sair): 56
O número secreto é menor.
Tentativas restantes: 6
Tentativa 2: Digite sua tentativa (01 a 99, ou 0 para sair): 34
O número secreto é menor.
Tentativas restantes: 5
Tentativa 3: Digite sua tentativa (01 a 99, ou 0 para sair): 22
O número secreto é menor.
Tentativas restantes: 4
Tentativa 4: Digite sua tentativa (01 a 99, ou 0 para sair): 11
O número secreto é menor.
Tentativas restantes: 3
Tentativa 5: Digite sua tentativa (01 a 99, ou 0 para sair): 03
O número secreto é maior.
Tentativas restantes: 2
Tentativa 6: Digite sua tentativa (01 a 99, ou 0 para sair): 08
Parabéns! Você acertou em 6 tentativas!
Digite seu nome para o recorde: Pedro
```

```
--- Recordes ---
1. Pedro - 6 tentativas
-----
```



DESENVOLVIMENTO DE GAMES (5º PERÍODO)

Minigame 1: Jogo do Marciano

Como Jogar

2. Desafio Final

- Um código secreto de 5 dígitos é gerado.
- Dois dos dígitos vêm do número que você descobriu anteriormente.
- Em cada tentativa, você digita um código de 5 números.
- O feedback visual funciona assim:
[n]: Dígito correto na posição correta;
(n): Dígito correto na posição errada;
n: Dígito não está no código.
- São permitidas até 8 tentativas para acertar o código.

```
O que você deseja fazer agora?
1. Jogar Novamente
2. Ir para o Desafio Final
3. Sair
Escolha uma opção: 2

--- Legenda do Desafio Final ---
No Desafio Final, o feedback da sua tentativa será dado da seguinte forma:
- Números entre parênteses '()' indicam que o dígito está presente no código, mas na posição incorreta.
- Números entre colchetes '[]' indicam que o dígito está presente e na posição correta.
- Números sem nenhum símbolo indicam que o dígito não está presente no código.
-----

--- Desafio Final ---
Você deve adivinhar a ordem correta de um código secreto de 5 dígitos.
Você já sabe que dois dos dígitos são: 0 e 8.
Você tem 8 tentativas.
Tentativas restantes: 7
Tentativa 1: Digite um código de 5 dígitos ou 'sair': 87091
Feedback: (8)7[0]9(1)
Tentativas restantes: 6
Tentativa 2: Digite um código de 5 dígitos ou 'sair': 13084
Feedback: [1]3[0](8)4
Tentativas restantes: 5
Tentativa 3: Digite um código de 5 dígitos ou 'sair': 18056
Feedback: [1](8)[0][5]6
Tentativas restantes: 4
Tentativa 4: Digite um código de 5 dígitos ou 'sair': 18052
Feedback: [1](8)[0][5](2)
Tentativas restantes: 3
Tentativa 5: Digite um código de 5 dígitos ou 'sair': 12058
Parabéns! Você acertou o código secreto!
-----

Você gostaria de jogar novamente? (s/n): [ ]
```

Código do minigame

```
import java.util.ArrayList;
import java.util.Collections;
import java.util.InputMismatchException;
import java.util.Random;
import java.util.Scanner;

public class marciano {

    static Scanner scanner = new Scanner(System.in);
    static Random random = new Random();
    static ArrayList<Recorde> recordes = new ArrayList<>();
    static final int TENTATIVAS_MAXIMAS = 8;
    static boolean jogadorAcertou = false;

    ...
```



DESENVOLVIMENTO DE GAMES (5º PERÍODO)

Minigame 1: Jogo do Marciano

Código do minigame

```
...
public static void main(String[] args) {
    introducaoDoJogo();
    boolean jogarNovamente = true;

    while (jogarNovamente) {
        jogadorAcertou = false;
        int numeroSecreto = jogarJogoDoMarciano();
        if (numeroSecreto != -1) {
            if (jogadorAcertou) {
                exibirMenuPosJogo(numeroSecreto);
            } else {
                System.out.println("Você não pode acessar o Desafio Final. Tente
novamente no próximo jogo.");
            }
        }
        jogarNovamente = solicitarNovoJogo();
    }

    System.out.println("Obrigado por jogar!");
}

static void introducaoDoJogo() {
    System.out.println("Bem-vindo ao Jogo do Marciano!");
    System.out.println("Em uma galáxia distante, um Marciano travesso escondeu um
número entre 01 e 99.");
    System.out.println("Sua missão é adivinhar esse número antes que ele fuja para
outra dimensão!");
    System.out.println("Você terá " + TENTATIVAS_MAXIMAS + " tentativas para
provar sua habilidade.");
    System.out.println("Boa sorte, Terráqueo!");
    System.out.println("-----");
}

static int jogarJogoDoMarciano() {
    int numeroSecreto = random.nextInt(99) + 1;
    int tentativas = 0;
    boolean acertou = false;

    System.out.println("\nNovo jogo! Adivinhe o número entre 01 e 99.");

    while (tentativas < TENTATIVAS_MAXIMAS && !acertou) {
        tentativas++;
        System.out.println("Tentativas restantes: " + (TENTATIVAS_MAXIMAS -
tentativas));
        int tentativa = obterTentativaDoJogador(tentativas);
        if (tentativa == -1) {
            return -1;
        }
    }
}
...
```



DESENVOLVIMENTO DE GAMES (5º PERÍODO)

Minigame 1: Jogo do Marciano

Código do minigame

```
...
    if (tentativa == numeroSecreto) {
        acertou = true;
        jogadorAcertou = true;
        System.out.println("Parabéns! Você acertou em " + tentativas + "
tentativas!");
        atualizarRecordes(tentativas);
    } else if (tentativa < numeroSecreto) {
        System.out.println("O número secreto é maior.");
    } else {
        System.out.println("O número secreto é menor.");
    }
}

if (!acertou) {
    System.out.println("Você não conseguiu adivinhar em " +
TENTATIVAS_MAXIMAS + " tentativas. O número era " + numeroSecreto + ".");
}

return numeroSecreto;
}

static int obterTentativaDoJogador(int tentativaNumero) {
    int tentativa;
    while (true) {
        try {
            System.out.print("Tentativa " + tentativaNumero + ": Digite sua tentativa (01
a 99, ou 0 para sair): ");
            tentativa = scanner.nextInt();
            scanner.nextLine();

            if (tentativa == 0) {
                return -1;
            }

            if (tentativa >= 1 && tentativa <= 99) {
                return tentativa;
            } else {
                System.out.println("Tentativa inválida. Digite um número entre 01 e 99.");
            }
        } catch (InputMismatchException e) {
            System.out.println("Entrada inválida. Digite um número inteiro.");
            scanner.nextLine();
        }
    }
}
...

```



DESENVOLVIMENTO DE GAMES (5º PERÍODO)

Minigame 1: Jogo do Marciano

Código do minigame

```
...

static void atualizarRecordes(int tentativas) {
    System.out.print("Digite seu nome para o recorde: ");
    String nome = scanner.nextLine();
    recordes.add(new Recorde(nome, tentativas));
    Collections.sort(recordes);
    exibirRecordes();
}

static void exibirRecordes() {
    System.out.println("\n--- Recordes ---");
    if (recordes.isEmpty()) {
        System.out.println("Nenhum recorde registrado ainda.");
    } else {
        for (int i = 0; i < Math.min(5, recordes.size()); i++) {
            System.out.println((i + 1) + ". " + recordes.get(i).nome + " - " +
recordes.get(i).tentativas + " tentativas");
        }
        System.out.println("-----");
    }
}

static void exibirMenuPosJogo(int numeroSecreto) {
    System.out.println("\nO que você deseja fazer agora?");
    System.out.println("1. Jogar Novamente");
    System.out.println("2. Ir para o Desafio Final");
    System.out.println("3. Sair");

    int escolha = obterEscolhaDoUsuario(1, 3);

    switch (escolha) {
        case 1:
            break;
        case 2:
            exibirLegendaDesafioFinal();
            desafioFinal(numeroSecreto);
            break;
        case 3:
            System.out.println("Obrigado por jogar!");
            System.exit(0);
            break;
    }
}

...
```



DESENVOLVIMENTO DE GAMES (5º PERÍODO)

Minigame 1: Jogo do Marciano

Código do minigame

```
...

static int obterEscolhaDoUsuario(int min, int max) {
    int escolha;
    while (true) {
        try {
            System.out.print("Escolha uma opção: ");
            escolha = scanner.nextInt();
            scanner.nextLine();

            if (escolha >= min && escolha <= max) {
                return escolha;
            } else {
                System.out.println("Opção inválida. Escolha um número entre " + min + " e " + max + ".");
            }
        } catch (InputMismatchException e) {
            System.out.println("Entrada inválida. Digite um número inteiro.");
            scanner.nextLine();
        }
    }
}

static boolean solicitarNovoJogo() {
    System.out.print("\nVocê gostaria de jogar novamente? (s/n): ");
    char resposta = scanner.next().toLowerCase().charAt(0);
    scanner.nextLine();
    return (resposta == 's');
}

static void exibirLegendaDesafioFinal() {
    System.out.println("\n--- Legenda do Desafio Final ---");
    System.out.println("No Desafio Final, o feedback da sua tentativa será dado da seguinte forma:");
    System.out.println(" - Números entre parênteses '()' indicam que o dígito está presente no código, mas na posição incorreta.");
    System.out.println(" - Números entre colchetes '[]' indicam que o dígito está presente e na posição correta.");
    System.out.println(" - Números sem nenhum símbolo indicam que o dígito não está presente no código.");
    System.out.println("-----");
}
...
```



DESENVOLVIMENTO DE GAMES (5º PERÍODO)

Minigame 1: Jogo do Marciano

Código do minigame

```
...

static void desafioFinal(int numeroJogoNormal) {
    System.out.println("\n--- Desafio Final ---");
    System.out.println("Você deve adivinhar a ordem correta de um código secreto de
5 dígitos.");
    int digito1 = numeroJogoNormal / 10;
    int digito2 = numeroJogoNormal % 10;
    System.out.println("Você já sabe que dois dos dígitos são: " + digito1 + " e " +
digito2 + ".");
    System.out.println("Você tem 8 tentativas.");

    ArrayList<Integer> codigoSecreto = gerarCodigoSecreto(numeroJogoNormal);
    int tentativas = 0;
    boolean acertou = false;

    while (tentativas < TENTATIVAS_MAXIMAS && !acertou) {
        tentativas++;
        System.out.println("Tentativas restantes: " + (TENTATIVAS_MAXIMAS -
tentativas));
        String tentativa = obterTentativaCodigo(tentativas);
        if (tentativa.equalsIgnoreCase("sair")) {
            System.out.println("Desafio Final encerrado.");
            return;
        }

        if (verificarTentativaCodigo(tentativa, codigoSecreto)) {
            acertou = true;
            System.out.println("Parabéns! Você acertou o código secreto!");
        } else {
            exibirFeedbackTentativa(tentativa, codigoSecreto, digito1, digito2);
        }
    }

    if (!acertou) {
        System.out.print("Você não conseguiu adivinhar o código secreto. O código era:
");
        for (int digito : codigoSecreto) {
            System.out.print(digito);
        }
        System.out.println();
    }
    System.out.println("-----");
}

...
```



DESENVOLVIMENTO DE GAMES (5º PERÍODO)

Minigame 1: Jogo do Marciano

Código do minigame

```
...
static ArrayList<Integer> gerarCodigoSecreto(int numeroJogoNormal) {
    ArrayList<Integer> digitosDisponiveis = new ArrayList<>();
    for (int i = 0; i <= 9; i++) {
        digitosDisponiveis.add(i);
    }

    int digito1 = numeroJogoNormal / 10;
    int digito2 = numeroJogoNormal % 10;

    digitosDisponiveis.remove(Integer.valueOf(digito1));
    digitosDisponiveis.remove(Integer.valueOf(digito2));

    ArrayList<Integer> codigoSecreto = new ArrayList<>(Collections.nCopies(5, -1));

    int posicaoDigito1 = random.nextInt(5);
    codigoSecreto.set(posicaoDigito1, digito1);

    int posicaoDigito2;
    do {
        posicaoDigito2 = random.nextInt(5);
    } while (posicaoDigito2 == posicaoDigito1);
    codigoSecreto.set(posicaoDigito2, digito2);

    for (int i = 0; i < 5; i++) {
        if (codigoSecreto.get(i) == -1) {
            int indice = random.nextInt(digitosDisponiveis.size());
            codigoSecreto.set(i, digitosDisponiveis.remove(indice));
        }
    }

    return codigoSecreto;
}

static String obterTentativaCodigo(int tentativaNumero) {
    String tentativa;
    while (true) {
        System.out.print("Tentativa " + tentativaNumero + ": Digite um código de 5
dígitos ou 'sair': ");
        tentativa = scanner.nextLine().trim();
        if (tentativa.equalsIgnoreCase("sair") || tentativa.matches("\\d{5}")) {
            return tentativa;
        }
        System.out.println("Código inválido. Digite exatamente 5 dígitos.");
    }
}
...
```



DESENVOLVIMENTO DE GAMES (5º PERÍODO)

Minigame 1: Jogo do Marciano

Código do minigame

```
...

    static boolean verificarTentativaCodigo(String tentativa, ArrayList<Integer>
codigoSecreto) {
    for (int i = 0; i < 5; i++) {
        if (Character.getNumericValue(tentativa.charAt(i)) != codigoSecreto.get(i)) {
            return false;
        }
    }
    return true;
}

    static void exibirFeedbackTentativa(String tentativa, ArrayList<Integer>
codigoSecreto, int digito1, int digito2) {
    StringBuilder feedback = new StringBuilder();
    for (int i = 0; i < 5; i++) {
        int digito = Character.getNumericValue(tentativa.charAt(i));
        if (digito == codigoSecreto.get(i)) {
            feedback.append("[").append(digito).append("]");
        } else if (codigoSecreto.contains(digito)) {
            feedback.append("(").append(digito).append(")");
        } else {
            feedback.append(digito);
        }
    }
    System.out.println("Feedback: " + feedback);
}

class Recorde implements Comparable<Recorde> {
    String nome;
    int tentativas;

    Recorde(String nome, int tentativas) {
        this.nome = nome;
        this.tentativas = tentativas;
    }

    public int compareTo(Recorde outro) {
        return Integer.compare(this.tentativas, outro.tentativas);
    }
}
```



DESENVOLVIMENTO DE GAMES (5º PERÍODO)

Minigame 2: Jogo da Velha

Descrição

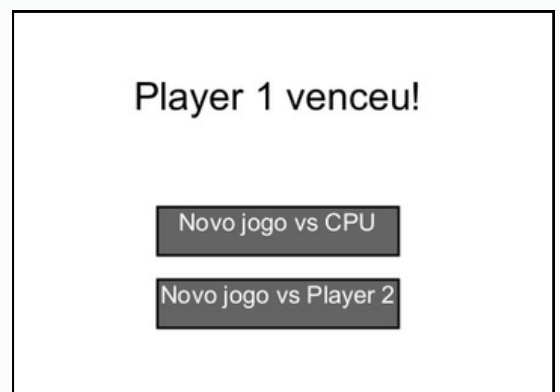
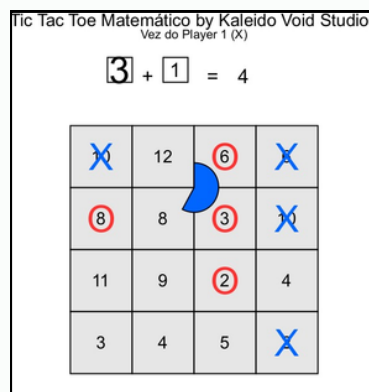
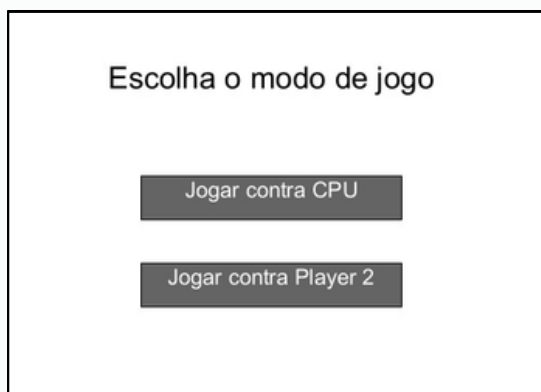
Tic Tac Toe Matemático é uma versão expandida do tradicional jogo da velha, agora com um tabuleiro 4x4 e uma mecânica baseada em soma de dados. Cada casa do tabuleiro possui um número de 2 a 12, e os jogadores só podem marcar casas compatíveis com a soma dos dados lançados ou pelo valor de somente um deles. É um jogo de estratégia, raciocínio e agilidade – tanto para jogar contra outro jogador quanto contra a CPU.

Diferenciais e Funcionalidades Principais da nossa versão

1. Casas do tabuleiro são ocupadas apenas se o número for igual a um dos dados ou à soma dos dois
2. Temporizador circular (formato pizza) que dá 8 segundos por turno
3. Jogador perde o turno se não clicar a tempo ou se não houver jogada válida
4. O jogo só reinicia após vitória, para permitir análise do resultado
5. Visual intuitivo com dados, símbolos e feedback colorido para cada jogador

Como Jogar

- Clique em "Jogar contra CPU" ou "2 Jogadores" no menu inicial.
- O jogador atual verá o botão "Rolar Dados".
- Após clicar, serão exibidos dois dados (ex: 4 e 5) e a soma deles (9).
- O jogador deve clicar em uma casa cujo valor seja igual ao primeiro dado, segundo dado ou à soma.
- Um temporizador em formato pizza começará a rodar após os dados serem rolados. O jogador precisa clicar antes que ele acabe.
- Se o jogador não clicar a tempo ou não houver jogada válida, ele perde o turno.
- O primeiro jogador a alinhar 4 marcas (X ou O) em linha, coluna ou diagonal vence.
- Após a vitória, será exibido o vencedor e duas opções: "Novo jogo contra CPU" ou "Novo jogo 2 jogadores".



Código do minigame

```
int[][] boardValues = new int[4][4];
char[][] boardMarks = new char[4][4];
boolean[][] marked = new boolean[4][4];
```

```
int currentPlayer = 1;
boolean isCPUMode = true;
boolean gameStarted = false;
boolean showEndScreen = false;
String winner = "";
```

```
...
```



Minigame 2: Jogo da Velha

Código do minigame

```
...
int die1, die2, diceSum;
boolean rolled = false;
boolean turnOver = false;
int timerStart;
int maxTime = 8000;

PFont font;
color player1Color = color(0, 102, 255);
color player2Color = color(0, 200, 0);
color cpuColor = color(255, 50, 50);

int cellSize = 100;
int margin = 50;
int boardX, boardY;

void setup() {
  size(600, 700);
  font = createFont("Arial", 32);
  textFont(font);
  boardX = width/2 - (cellSize*2);
  boardY = 200;
  resetBoard();
}

void draw() {
  background(255);

  if (!gameStarted && !showEndScreen) {
    showMenu();
    return;
  }

  if (showEndScreen) {
    showEndGameScreen();
    return;
  }

  drawTitle();
  drawBoard();
  drawDiceInfo();
  drawTimer();
  drawTurn();

  if (rolled && millis() - timerStart > maxTime && !turnOver) {
    println("Tempo esgotado!");
    passTurn();
  }
}
```

...



Minigame 2: Jogo da Velha

Código do minigame

...

```
if (rolled && currentPlayer == 2 && isCPUMode && !turnOver) {  
    cpuPlay();  
}
```

```
    checkWin();  
}
```

```
void resetBoard() {  
    ArrayList<Integer> pool = new ArrayList<Integer>();  
    for (int i = 2; i <= 12; i++) {  
        int count = (i == 11 || i == 12) ? 1 : 2;  
        for (int j = 0; j < count; j++) {  
            pool.add(i);  
        }  
    }  
    java.util.Collections.shuffle(pool);  
    int idx = 0;  
    for (int i = 0; i < 4; i++) {  
        for (int j = 0; j < 4; j++) {  
            boardValues[i][j] = pool.get(idx++);  
            boardMarks[i][j] = ' ';  
            marked[i][j] = false;  
        }  
    }  
}
```

```
void drawBoard() {  
    strokeWeight(2);  
    for (int i = 0; i < 4; i++) {  
        for (int j = 0; j < 4; j++) {  
            int x = boardX + j * cellSize;  
            int y = boardY + i * cellSize;  
            fill(230);  
            rect(x, y, cellSize, cellSize);  
            fill(0);  
            textSize(28);  
            textAlign(CENTER, CENTER);  
            text(boardValues[i][j], x + cellSize/2, y + cellSize/2);  
        }  
    }  
}
```

```
if (marked[i][j]) {  
    fill(currentMarkColor(boardMarks[i][j]));  
    textSize(60);  
    text(boardMarks[i][j], x + cellSize/2, y + cellSize/2);  
}  
}  
}  
}
```

...



Minigame 2: Jogo da Velha

Código do minigame

```
...
color currentMarkColor(char mark) {
    if (mark == 'X') return player1Color;
    if (isCPUMode) return cpuColor;
    return player2Color;
}

void drawDiceInfo() {
    if (!rolled) return;

    int x = width/2 - 140;
    int y = 120;
    drawDie(die1, x);
    textSize(32);
    fill(0);
    text("+", x + 65, y);
    drawDie(die2, x + 90);
    text("=", x + 170, y);
    text(diceSum, x + 220, y);
}

void drawDie(int val, int x) {
    int y = 90;
    fill(255);
    rect(x, y, 40, 40);
    fill(0);
    textAlign(CENTER, CENTER);
    text(val, x + 20, y + 20);
}

void drawTimer() {
    if (!rolled) return;

    int timePassed = millis() - timerStart;
    float angle = map(timePassed, 0, maxTime, 0, TWO_PI);
    pushMatrix();
    translate(width/2, 300);
    fill(currentPlayer == 1 ? player1Color : (isCPUMode ? cpuColor : player2Color));
    arc(0, 0, 80, 80, -HALF_PI, -HALF_PI + TWO_PI - angle, PIE);
    popMatrix();
}

void drawTitle() {
    fill(0);
    textAlign(CENTER, CENTER);
    textSize(28);
    text("Tic Tac Toe Matemático by Kaleido Void Studios", width/2, 30);
}
...
```



DESENVOLVIMENTO DE GAMES (5º PERÍODO)

Minigame 2: Jogo da Velha

Código do minigame

```
...
void drawTurn() {
    textSize(20);
    fill(0);
    textAlign(CENTER);
    String txt = "Vez do " + (currentPlayer == 1 ? "Player 1 (X)" : (isCPUMode ? "CPU (O)" :
"Player 2 (O)"));
    text(txt, width/2, 60);
    if (!rolled) {
        fill(100);
        rect(width/2 - 60, 70, 120, 30);
        fill(255);
        text("Rolar Dados", width/2, 85);
    }
}

void mousePressed() {
    if (!gameStarted) {
        if (mouseY > height/2 - 20 && mouseY < height/2 + 20) {
            if (mouseX > width/2 - 120 && mouseX < width/2 + 120) {
                isCPUMode = true;
                gameStarted = true;
            }
        }
        if (mouseY > height/2 + 60 && mouseY < height/2 + 100) {
            if (mouseX > width/2 - 120 && mouseX < width/2 + 120) {
                isCPUMode = false;
                gameStarted = true;
            }
        }
    }
    return;
}

if (showEndScreen) {
    if (mouseX > width/2 - 100 && mouseX < width/2 + 100) {
        if (mouseY > height/2 + 20 && mouseY < height/2 + 60) {
            isCPUMode = true;
            restartGame();
        } else if (mouseY > height/2 + 80 && mouseY < height/2 + 120) {
            isCPUMode = false;
            restartGame();
        }
    }
}
return;
}

...
```



DESENVOLVIMENTO DE GAMES (5º PERÍODO)

Minigame 2: Jogo da Velha

Código do minigame

```
...
if (!rolled) {
    if (mouseX > width/2 - 60 && mouseX < width/2 + 60 &&
        mouseY > 70 && mouseY < 100) {
        rollDice();
    }
    return;
}

if (turnOver || (currentPlayer == 2 && isCPUMode)) return;

for (int i = 0; i < 4; i++) {
    for (int j = 0; j < 4; j++) {
        int x = boardX + j * cellSize;
        int y = boardY + i * cellSize;
        if (mouseX > x && mouseX < x + cellSize &&
            mouseY > y && mouseY < y + cellSize &&
            !marked[i][j]) {
            int val = boardValues[i][j];
            if (val == die1 || val == die2 || val == diceSum) {
                boardMarks[i][j] = currentPlayer == 1 ? 'X' : 'O';
                marked[i][j] = true;
                turnOver = true;
            }
        }
    }
}

if (turnOver) passTurn();
}

void rollDice() {
    die1 = int(random(1, 7));
    die2 = int(random(1, 7));
    diceSum = die1 + die2;
    rolled = true;
    timerStart = millis();
    turnOver = false;
}

void passTurn() {
    rolled = false;
    turnOver = false;
    currentPlayer = 3 - currentPlayer;
}

void cpuPlay() {
    delay(1000);
    for (int i = 0; i < 4; i++) {
        for (int j = 0; j < 4; j++) {
            if (!marked[i][j] &&
...

```



DESENVOLVIMENTO DE GAMES (5º PERÍODO)

Minigame 2: Jogo da Velha

Código do minigame

```
...
passTurn();
}

void checkWin() {
    for (int i = 0; i < 4; i++) {
        if (checkLine(boardMarks[i][0], boardMarks[i][1], boardMarks[i][2], boardMarks[i][3]))
        {
            endGame(boardMarks[i][0]);
            return;
        }
        if (checkLine(boardMarks[0][i], boardMarks[1][i], boardMarks[2][i], boardMarks[3][i]))
        {
            endGame(boardMarks[0][i]);
            return;
        }
    }
    if (checkLine(boardMarks[0][0], boardMarks[1][1], boardMarks[2][2], boardMarks[3][3])) {
        endGame(boardMarks[0][0]);
        return;
    }
    if (checkLine(boardMarks[0][3], boardMarks[1][2], boardMarks[2][1], boardMarks[3][0])) {
        endGame(boardMarks[0][3]);
        return;
    }
}

boolean checkLine(char a, char b, char c, char d) {
    return a != ' ' && a == b && b == c && c == d;
}

void endGame(char winnerMark) {
    showEndScreen = true;
    if (winnerMark == 'X') winner = "Player 1 venceu!";
    else if (isCPUMode) winner = "CPU venceu!";
    else winner = "Player 2 venceu!";
}

void showEndGameScreen() {
    background(255);
    fill(0);
    textAlign(CENTER);
    textSize(32);
    text(winner, width/2, height/2 - 60);
}

...
```



DESENVOLVIMENTO DE GAMES (5º PERÍODO)

Minigame 2: Jogo da Velha

Código do minigame

```
fill(100);
rect(width/2 - 100, height/2 + 20, 200, 40);
fill(255);
textSize(20);
text("Novo jogo vs CPU", width/2, height/2 + 40);

fill(100);
rect(width/2 - 100, height/2 + 80, 200, 40);
fill(255);
text("Novo jogo vs Player 2", width/2, height/2 + 100);
}

void showMenu() {
  background(255);
  fill(0);
  textAlign(CENTER);
  textSize(28);
  text("Escolha o modo de jogo", width/2, height/2 - 100);

  fill(100);
  rect(width/2 - 120, height/2 - 20, 240, 40);
  fill(255);
  textSize(20);
  text("Jogar contra CPU", width/2, height/2);

  fill(100);
  rect(width/2 - 120, height/2 + 60, 240, 40);
  fill(255);
  text("Jogar contra Player 2", width/2, height/2 + 80);
}

void restartGame() {
  resetBoard();
  rolled = false;
  showEndScreen = false;
  currentPlayer = 1;
}
```



DESENVOLVIMENTO DE GAMES (5º PERÍODO)

Minigame 3: Jogo da Forca

Descrição

Este é um jogo da força clássico implementado usando a ferramenta Processing. O jogador tenta adivinhar uma palavra secreta, letra por letra, com a ajuda de uma dica. A cada tentativa incorreta, uma parte do desenho da força é adicionada. O jogador vence se adivinhar a palavra antes que o desenho da força seja completo.



Diferenciais e Funcionalidades Principais da nossa versão

1. Seleção Aleatória de Palavras e Dicas: O jogo escolhe aleatoriamente uma palavra secreta de uma lista predefinida e fornece uma dica correspondente.
2. Interface Gráfica Simples: A interface do jogo é criada usando as funções de desenho do Processing para exibir o estado da palavra, o desenho da força, o número de tentativas restantes e as letras chutadas.
3. Controle de Tentativas: O jogador tem um número limitado de tentativas incorretas (neste caso, 6).
4. Feedback Visual: O jogo atualiza a exibição da palavra secreta conforme letras corretas são adivinhadas, mostrando underscores para letras desconhecidas. Ele também desenha partes da força para cada tentativa incorreta.
5. Registro de Letras Chutadas: O jogo mantém o controle das letras que o jogador já tentou adivinhar e as exibe na tela.
6. Dica da Palavra: Uma dica relacionada à palavra secreta é exibida na tela para ajudar o jogador. A dica é geralmente mais longa que a palavra secreta e é exibida em azul marinho.
7. Detecção de Vitória e Derrota: O jogo verifica se o jogador adivinhou todas as letras da palavra secreta (vitória) ou se excedeu o número máximo de tentativas incorretas (derrota).
8. Tela Final do Jogo: Ao final do jogo, uma tela preenche toda a janela com fundo verde claro e texto azul marinho, informando se o jogador venceu ou perdeu e qual era a palavra secreta em caso de derrota.
9. Jogar Novamente: Após o término de uma partida, o jogador pode pressionar qualquer tecla para iniciar uma nova rodada com uma palavra secreta diferente.



DESENVOLVIMENTO DE GAMES (5º PERÍODO)

Minigame 3: Jogo da Forca

Código do minigame

```
String[] wordList = {"barcelona", "riachuelo", "hipismo", "ampulheta", "madagascar"};
String[] wordHints = {
    "Time europeu",
    "Loja de roupas",
    "Esporte olímpico",
    "Objeto",
    "País"
};
String secretWord;
String wordHint;
char[] guessedWordDisplay;
boolean[] guessedLetters;
int incorrectGuesses = 0;
int maxGuesses = 6;
boolean gameOver = false;
String gameResult = "";

color backgroundColor = color(204, 255, 204);
color hangmanColor = color(0);
color textColor = color(0, 0, 102);
color hintColor = color(0, 0, 102);
color resultTextColor = color(0, 0, 102);
color resultBackgroundColor = color(204, 255, 204);

void setup() {
    size(600, 400);
    startGame();
}

void draw() {
    background(backgroundColor);
    drawHangman();
    displayWord();
    displayGuesses();
    displayHint(); // Adiciona a exibição da dica
    checkGameOver();

    if (gameOver) {
        displayFinalScreen();
    }
}

void startGame() {
    selectWord();
    initializeGame();
}

void selectWord() {
    int randomIndex = floor(random(wordList.length));
    secretWord = wordList[randomIndex].toUpperCase();
    wordHint = wordHints[randomIndex];
}

...
```



DESENVOLVIMENTO DE GAMES (5º PERÍODO)

Minigame 3: Jogo da Forca

Código do minigame

```
void initializeGame() {
    guessedWordDisplay = new char[secretWord.length()];
    guessedLetters = new boolean[26]; // A-Z
    for (int i = 0; i < secretWord.length(); i++) {
        guessedWordDisplay[i] = '_';
    }
    incorrectGuesses = 0;
    gameOver = false;
    gameResult = "";
}
```

```
void drawHangman() {
    stroke(hangmanColor);
    strokeWeight(2);
    // Base
    line(100, 350, 200, 350);
    line(150, 350, 150, 100);
    line(150, 100, 250, 100);
    line(250, 100, 250, 150);

    if (incorrectGuesses > 0) {
        // Cabeça
        ellipse(250, 175, 25, 25);
    }
    if (incorrectGuesses > 1) {
        // Corpo
        line(250, 200, 250, 275);
    }
    if (incorrectGuesses > 2) {
        // Braço esquerdo
        line(250, 220, 220, 240);
    }
    if (incorrectGuesses > 3) {
        // Braço direito
        line(250, 220, 280, 240);
    }
    if (incorrectGuesses > 4) {
        // Perna esquerda
        line(250, 275, 220, 295);
    }
    if (incorrectGuesses > 5) {
        // Perna direita
        line(250, 275, 280, 295);
    }
}
```

...



DESENVOLVIMENTO DE GAMES (5º PERÍODO)

Minigame 3: Jogo da Forca

Código do minigame

```
void displayWord() {
    fill(textColor);
    textSize(32);
    textAlign(LEFT); // Alinha o texto à esquerda
    text(new String(guessedWordDisplay), 300, 150); // Exibe a palavra à direita da forca
}
void displayGuesses() {
    fill(textColor);
    textSize(16);
    textAlign(LEFT);
    text("Tentativas restantes: " + (maxGuesses - incorrectGuesses), 300, 200);
    String guessed = "Letras chutadas: ";
    for (int i = 0; i < 26; i++) {
        if (guessedLetters[i]) {
            guessed += char(i + 'A') + " ";
        }
    }
    text(guessed, 300, 230);
}
void displayHint() {
    fill(hintColor);
    textSize(20); // Dica maior
    textAlign(LEFT);
    text("Dica: " + wordHint, 300, 100); // Exibe a dica
}
void checkGameOver() {
    boolean won = true;
    for (char c : guessedWordDisplay) {
        if (c == ' ') {
            won = false;
            break;
        }
    }
}
if (won) {
    gameOver = true;
    gameResult = "Você venceu!";
} else if (incorrectGuesses >= maxGuesses) {
    gameOver = true;
    gameResult = "Você perdeu! A palavra era: " + secretWord;
}
}
void displayFinalScreen() {
    background(resultBackgroundColor);
    fill(resultTextColor);
    textSize(24);
    textAlign(CENTER, CENTER);
    text(gameResult, width / 2, height / 2);
    textSize(16);
    text("Pressione qualquer tecla para jogar novamente",
    width / 2, height / 2 + 30);
}
...
```



DESENVOLVIMENTO DE GAMES (5º PERÍODO)

Minigame 3: Jogo da Forca

Código do minigame

```
void keyPressed() {
  if (gameOver) {
    startGame(); // Inicia um novo jogo com uma palavra diferente
  } else {
    char guess = char(keyCode);
    if (guess >= 'A' && guess <= 'Z') {
      if (!guessedLetters[guess - 'A']) {
        guessedLetters[guess - 'A'] = true;
        boolean correctGuess = false;
        for (int i = 0; i < secretWord.length(); i++) {
          if (secretWord.charAt(i) == guess) {
            guessedWordDisplay[i] = guess;
            correctGuess = true;
          }
        }
        if (!correctGuess) {
          incorrectGuesses++;
        }
      }
    }
  }
}
```



DESENVOLVIMENTO DE GAMES (5º PERÍODO)

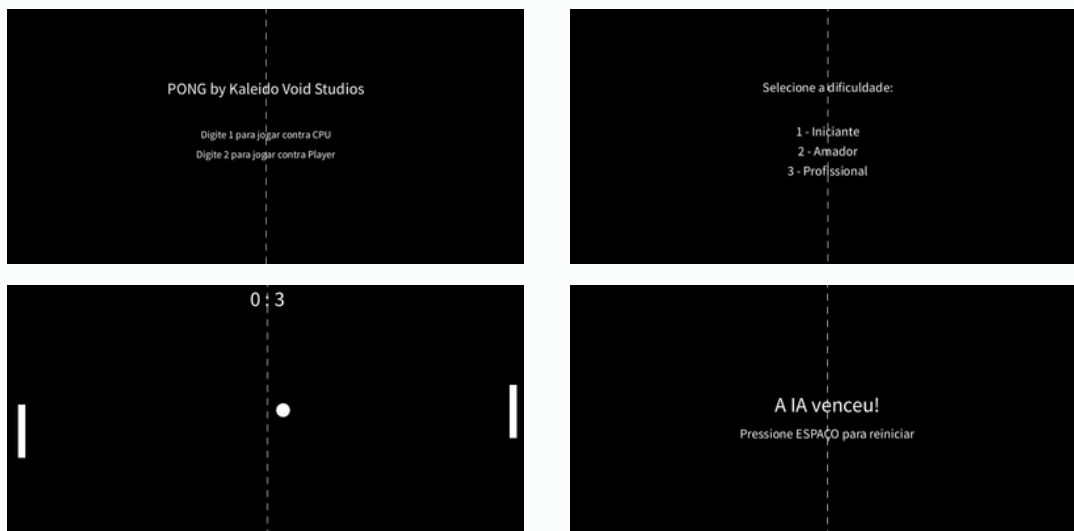
Minigame 4: Pong

Descrição

Pong by Kaleido Void Studios é uma releitura moderna e divertida do clássico Pong, com modos singleplayer e multiplayer local. Com visual retrô e jogabilidade afiada, o jogo foi redesenhado com mecânicas inteligentes, IA com diferentes níveis de dificuldade e um sistema de pontuação objetivo e competitivo. Ideal para partidas rápidas e desafiadoras entre amigos ou contra o computador.

Diferenciais e Funcionalidades Principais da nossa versão

1. IA com 3 níveis de dificuldade ajustável.
2. Visual retrô com efeitos inspirados no arcade original.
3. Modo multiplayer local para jogar com amigos.
4. Pontuação fixa em 7 pontos – rápido, competitivo e dinâmico.
5. "Preparar?" (READY) aparece apenas antes da primeira rodada, garantindo fluidez nos próximos pontos.
6. Física da bola com variações aleatórias na colisão, deixando a partida imprevisível.
7. Aumento de velocidade a cada colisão, criando partidas emocionantes!



Como Jogar

- JOGADOR 1 (lado esquerdo):
 - Sobe: tecla 'W'
 - Desce: tecla 'S'
- JOGADOR 2 (lado direito):
 - Sobe: tecla '↑' (seta para cima)
 - Desce: tecla '↓' (seta para baixo)
- Início do Jogo:
 - No menu inicial, pressione:
 - '1' para jogar contra a IA (CPU)
 - '2' para jogar contra outro jogador local
- Dificuldade da IA (modo singleplayer):
 - Após escolher o modo 1, pressione:
 - '1' para Fácil
 - '2' para Médio
 - '3' para Difícil
- Após marcar o primeiro ponto, o jogo continua automaticamente.
- O jogo termina quando um dos jogadores atinge 7 pontos.
- Pressione ESPAÇO para iniciar a rodada inicial ou reiniciar após o fim do jogo.



DESENVOLVIMENTO DE GAMES (5º PERÍODO)

Minigame 4: Pong

Código do minigame

```
enum GameState { MENU, SELECT_DIFFICULTY, READY, PLAYING, GAME_OVER }  
GameState gameState = GameState.MENU;
```

```
int score1 = 0, score2 = 0;  
int winningScore = 7;
```

```
float paddle1Y, paddle2Y;  
float paddleWidth = 10, paddleHeight = 80;  
float paddleSpeed = 5;
```

```
float ballX, ballY;  
float ballSize = 20;  
float ballSpeedX, ballSpeedY;
```

```
boolean singlePlayer = true;  
int difficulty = 1;
```

```
boolean roundStarted = false;
```

```
void setup() {  
  size(800, 400);  
  resetGame();  
}
```

```
void draw() {  
  background(0);  
  drawMiddleLine();
```

```
  switch (gameState) {  
    case MENU:  
      drawMenu();  
      break;  
    case SELECT_DIFFICULTY:  
      drawDifficultySelection();  
      break;  
    case READY:  
      drawReady();  
      break;  
    case PLAYING:  
      drawGame();  
      updateBall();  
      break;  
    case GAME_OVER:  
      drawGameOver();  
      break;
```

```
  }  
}
```

```
...
```



DESENVOLVIMENTO DE GAMES (5º PERÍODO)

Minigame 4: Pong

Código do minigame

```
void drawMiddleLine() {
    stroke(255);
    for (int y = 0; y < height; y += 20) {
        line(width/2, y, width/2, y+10);
    }
}

void drawMenu() {
    fill(255);
    textAlign(CENTER);
    textSize(24);
    text("PONG by Kaleido Void Studios", width/2, height/3);
    textSize(16);
    text("Digite 1 para jogar contra CPU", width/2, height/2);
    text("Digite 2 para jogar contra Player", width/2, height/2 + 30);
}

void drawDifficultySelection() {
    fill(255);
    textAlign(CENTER);
    textSize(20);
    text("Selecione a dificuldade:", width/2, height/3);
    text("1 - Iniciante", width/2, height/2);
    text("2 - Amador", width/2, height/2 + 30);
    text("3 - Profissional", width/2, height/2 + 60);
}

void drawReady() {
    fill(255);
    textAlign(CENTER);
    textSize(32);
    text("Preparar?", width/2, height/2);
}

void drawGame() {
    fill(255);
    rect(20, paddle1Y, paddleWidth, paddleHeight);
    rect(width - 20 - paddleWidth, paddle2Y, paddleWidth, paddleHeight);
    ellipse(ballX, ballY, ballSize, ballSize);
    textSize(32);
    textAlign(CENTER);
    text(score1 + " : " + score2, width/2, 40);

    movePaddles();
}

...
```



DESENVOLVIMENTO DE GAMES (5º PERÍODO)

Minigame 4: Pong

Código do minigame

```
void drawGameOver() {
    fill(255);
    textAlign(CENTER);
    textSize(32);
    String winner = score1 > score2 ? "Player 1 venceu!" : "Player 2 venceu!";
    if (singlePlayer) winner = score1 > score2 ? "Você venceu!" : "A IA venceu!";
    text(winner, width/2, height/2);
    textSize(20);
    text("Pressione ESPAÇO para reiniciar", width/2, height/2 + 40);
}

void movePaddles() {
    if (keyPressed) {
        if (key == 'w' || key == 'W') {
            paddle1Y -= paddleSpeed;
        }
        if (key == 's' || key == 'S') {
            paddle1Y += paddleSpeed;
        }
    }

    if (singlePlayer) {
        moveAI();
    } else {
        if (keyPressed) {
            if (keyCode == UP) {
                paddle2Y -= paddleSpeed;
            }
            if (keyCode == DOWN) {
                paddle2Y += paddleSpeed;
            }
        }
    }

    paddle1Y = constrain(paddle1Y, 0, height - paddleHeight);
    paddle2Y = constrain(paddle2Y, 0, height - paddleHeight);
}

void moveAI() {
    float target = ballY - paddleHeight / 2;
    float aiSpeed = 2;
    if (difficulty == 2) aiSpeed = 3.5;
    if (difficulty == 3) aiSpeed = 5;

    if (paddle2Y < target) paddle2Y += aiSpeed;
    else paddle2Y -= aiSpeed;
    paddle2Y = constrain(paddle2Y, 0, height - paddleHeight);
}

void updateBall() {
    if (!roundStarted) return;
    ...
}
```



DESENVOLVIMENTO DE GAMES (5º PERÍODO)

Minigame 4: Pong

Código do minigame

```
ballX += ballSpeedX;
ballY += ballSpeedY;

if (ballY < 0 || ballY > height) ballSpeedY *= -1;

if (ballX - ballSize/2 < 30 && ballY > paddle1Y && ballY < paddle1Y +
paddleHeight) {
    ballSpeedX *= -1.1;
    ballSpeedY = random(-4, 4);
}

if (ballX + ballSize/2 > width - 30 && ballY > paddle2Y && ballY < paddle2Y +
paddleHeight) {
    ballSpeedX *= -1.1;
    ballSpeedY = random(-4, 4);
}

if (ballX < 0) {
    score2++;
    checkWin();
} else if (ballX > width) {
    score1++;
    checkWin();
}
}

void checkWin() {
    if (score1 >= winningScore || score2 >= winningScore) {
        gameState = GameState.GAME_OVER;
        roundStarted = false;
    } else {
        resetRound();
    }
}

void resetGame() {
    paddle1Y = paddle2Y = height / 2 - paddleHeight / 2;
    score1 = score2 = 0;
    roundStarted = false;
    gameState = GameState.MENU;
}

void resetRound() {
    ballX = width/2;
    ballY = height/2;
    roundStarted = false;
}
...
```



Minigame 4: Pong

Código do minigame

```
if (score1 == 0 && score2 == 0) {
    ballSpeedX = 0;
    ballSpeedY = 0;
    gameState = GameState.READY;
} else {
    startGame();
}

void startGame() {
    roundStarted = true;
    gameState = GameState.PLAYING;
    ballSpeedX = random(1) < 0.5 ? 4 : -4;
    ballSpeedY = random(-3, 3);
}

void keyPressed() {
    if (gameState == GameState.MENU) {
        if (key == '1') {
            singlePlayer = true;
            gameState = GameState.SELECT_DIFFICULTY;
        } else if (key == '2') {
            singlePlayer = false;
            gameState = GameState.READY;
        }
    } else if (gameState == GameState.SELECT_DIFFICULTY) {
        if (key >= '1' && key <= '3') {
            difficulty = int(key) - 48;
            gameState = GameState.READY;
        }
    } else if (gameState == GameState.READY) {
        if (key == ' ') {
            startGame();
        }
    } else if (gameState == GameState.GAME_OVER) {
        if (key == ' ') {
            resetGame();
        }
    }
}
```

DESENVOLVIMENTO DE GAMES (5º PERÍODO)

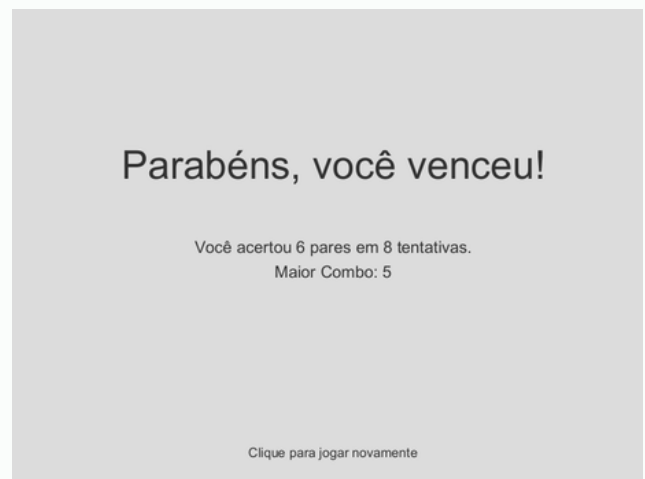
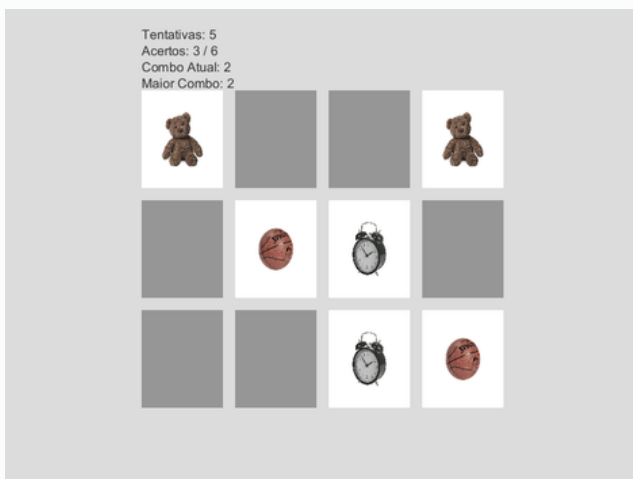
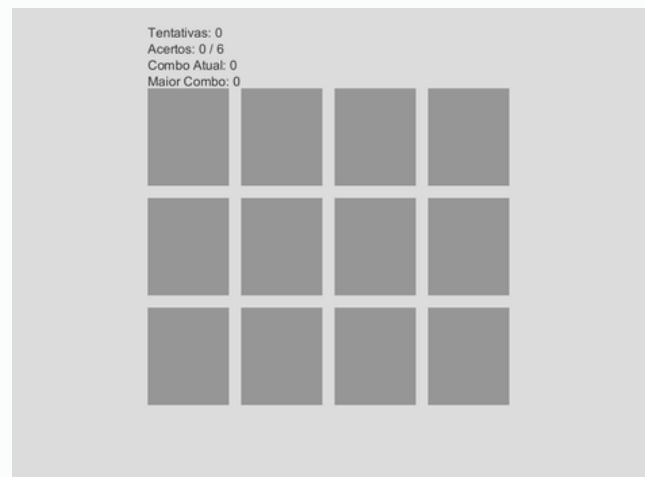
Minigame 5: Jogo da Memória

Descrição

Um divertido e educativo jogo da memória desenvolvido em Processing, onde o objetivo é encontrar pares de cartas iguais. Com temas visuais atrativos como frutas, objetos e personagens, o jogo proporciona um excelente exercício de memória e concentração para todas as idades.

Diferenciais e Funcionalidades Principais da nossa versão

1. Geração automática e embaralhada de cartas pares;
2. Seletor de tema antes do início do jogo;
3. Contagem de tentativas e acertos;
4. Controle de cartas viradas e encontradas;
5. Delay visual ao errar um par para melhor feedback;
6. Detecção de fim de jogo com mensagem de parabéns;
7. Medição de combos e registro do maior combo da sessão.



Como Jogar

- Selecione um tema entre frutas, objetos ou personagens.
- Clique no botão "Iniciar Jogo".
- Clique nas cartas para virá-las.
- Tente encontrar os pares com o menor número de tentativas.
- Acertos consecutivos aumentam seu combo atual.
- Se errar um par, o combo é resetado.
- Quando todos os pares forem encontrados, o jogo será encerrado automaticamente.
- Clique na tela para reiniciar o jogo após vencer.



DESENVOLVIMENTO DE GAMES (5º PERÍODO)

Minigame 5: Jogo da Memória

Código do minigame

```
int numPares = 6;
int numCartas = numPares * 2;
int[] cartas;
boolean[] viradas;
boolean[] encontradas;
int cartaSelecionada1 = -1;
int cartaSelecionada2 = -1;
int tentativas = 0;
int acertos = 0;
int comboAtual = 0; // << Novo: combo atual
int maiorCombo = 0; // << Novo: maior combo alcançado
int larguraCarta = 100;
int alturaCarta = 120;
PImage[] imagens;
PImage versoCarta;
String[] temas = {"frutas", "objetos", "personagens"};
String temaAtual = "frutas";
boolean jogoIniciado = false;
PFont fonteTitulo;
PFont fonteInstrucao;
PFont fonteTema;
color corFundo = color(220);
color corTexto = color(50);
color corBotao = color(100, 150, 200);
color corTextoBotao = color(255);
int botaoLargura = 150;
int botaoAltura = 40;
int botaoX;
int botaoY;
boolean jogoFinalizado = false;
boolean aguardandoPausa = false;
int tempoPausa = 1000;
int tempoVirada = 0;
```

```
String[][] linksTemas = {
  {
    "https://i.pinimg.com/736x/7d/21/ba/7d21bac43eab86e78d35454f664029c7.jpg",
    "https://i.pinimg.com/736x/6f/0c/b2/6f0cb27d1fd2257f841ee752f6d99f2d.jpg",
    "https://i.pinimg.com/736x/d5/b1/ee/d5b1ee043c6742daeb07f5d12d48591b.jpg",
    "https://i.pinimg.com/736x/10/66/e2/1066e21e625a23571fe52b85568739ac.jpg",
    "https://i.pinimg.com/736x/77/be/4c/77be4c593d50eaa9f1718103d29b203d.jpg",
    "https://i.pinimg.com/736x/f5/86/dc/f586dc149657d79f1b70b90553b3d2f1.jpg"
  },
  {
    "https://i.pinimg.com/736x/cc/44/a3/cc44a35b3d2b18a713c6938ee8d44e6a.jpg",
    "https://i.pinimg.com/736x/87/81/d6/8781d63e3eca2e5d8dc0fa6dfba3eb6c.jpg",
    "https://i.pinimg.com/736x/3d/90/6f/3d906f3c6794c81f60124687bb9d46c4.jpg",
    "https://i.pinimg.com/736x/3d/9a/a3/3d9aa3e0f0b9b9ff359b14f76e1ee780.jpg",
    "https://i.pinimg.com/736x/66/c0/52/66c052726391706e5b4c667e1e500dff.jpg",
    "https://i.pinimg.com/736x/29/1b/57/291b571a8ea862a2f0e6037d1a79ebb8.jpg"
  },
  ...
}
```



DESENVOLVIMENTO DE GAMES (5º PERÍODO)

Minigame 5: Jogo da Memória

Código do minigame

```
{
  "https://i.pinimg.com/736x/08/ef/29/08ef29666fa9f82c8e80f3107f5b100a.jpg",
  "https://i.pinimg.com/736x/10/fc/ca/10fccaffa90ff7b6aeade229a0756ab5.jpg",
  "https://i.pinimg.com/736x/6a/25/f6/6a25f64b9b831504a797d27dcfe53330.jpg",
  "https://i.pinimg.com/736x/14/0c/24/140c248717e6a79b4eee30ef63e1e7d1.jpg",
  "https://i.pinimg.com/736x/56/a6/22/56a6220b5cd554d9ff00941025cf67cd.jpg",
  "https://i.pinimg.com/736x/00/e0/88/00e088ee0818afded34d683a5dbf4c90.jpg"
}
};

void setup() {
  size(800, 600);
  fonteTitulo = createFont("Arial", 48, true);
  fonteInstrucao = createFont("Arial", 16, true);
  fonteTema = createFont("Arial", 20, true);
  textAlign(CENTER, CENTER);
  versoCarta = createImage(larguraCarta, alturaCarta, RGB);
  versoCarta.loadPixels();
  for (int i = 0; i < versoCarta.pixels.length; i++) versoCarta.pixels[i] = color(150);
  versoCarta.updatePixels();
  carregarTema(temaAtual);
  inicializarJogo();
  botaoX = width / 2 - botaoLargura / 2;
  botaoY = height - 100;
}

void draw() {
  background(corFundo);

  if (!jogoIniciado) {
    textFont(fonteTitulo);
    fill(corTexto);
    text("Jogo da Memória", width / 2, height / 4);
    textFont(fonteInstrucao);
    text("Selecione um tema e clique em Iniciar", width / 2, height / 3);
    float yTema = height / 2 - 40;
    for (String tema : temas) {
      fill(corTexto);
      if (tema.equals(temaAtual)) fill(255, 100, 100);
      textFont(fonteTema);
      text(tema, width / 2, yTema);
      yTema += 30;
    }
    fill(corBotao);
    rect(botaoX, botaoY, botaoLargura, botaoAltura, 10);
    textFont(fonteInstrucao);
    fill(corTextoBotao);
    text("Iniciar Jogo", width / 2, botaoY + botaoAltura / 2);
    return;
  }
  ...
}
```



DESENVOLVIMENTO DE GAMES (5º PERÍODO)

Minigame 5: Jogo da Memória

Código do minigame

```
if (jogoFinalizado) {
    textFont(fonteTitulo);
    fill(corTexto);
    text("Parabéns, você venceu!", width / 2, height / 3);
    textFont(fonteTema);
    text("Você acertou " + acertos + " pares em " + tentativas + " tentativas.", width / 2,
height / 2);
    text("Maior Combo: " + maiorCombo, width / 2, height / 2 + 30);
    textFont(fonteInstrucao);
    text("Clique para jogar novamente", width / 2, height - 50);
    return;
}
int espacoHorizontal = 15;
int espacoVertical = 15;
int cartasPorLinha = 4;
int numLinhas = (int) ceil((float) numCartas / cartasPorLinha);
float larguraTotalGrid = (cartasPorLinha * larguraCarta) + ((cartasPorLinha - 1) *
espacoHorizontal);
float alturaTotalGrid = (numLinhas * alturaCarta) + ((numLinhas - 1) * espacoVertical);
float margemLateral = (width - larguraTotalGrid) / 2;
float margemSuperior = (height - alturaTotalGrid) / 2;
for (int i = 0; i < numCartas; i++) {
    int coluna = i % cartasPorLinha;
    int linha = i / cartasPorLinha;
    int x = (int) (coluna * (larguraCarta + espacoHorizontal) + larguraCarta / 2 +
margemLateral);
    int y = (int) (linha * (alturaCarta + espacoVertical) + alturaCarta / 2 +
margemSuperior);
    if (encontradas[i] || viradas[i]) {
        image(imagens[cartas[i]], x - larguraCarta / 2, y - alturaCarta / 2, larguraCarta,
alturaCarta);
    } else {
        image(versoCarta, x - larguraCarta / 2, y - alturaCarta / 2, larguraCarta,
alturaCarta);
    }
}
textFont(fonteInstrucao);
fill(corTexto);
textAlign(LEFT, TOP);
text("Tentativas: " + tentativas, (int)margemLateral, 30);
text("Acertos: " + acertos + " / " + numPares, (int)margemLateral, 50);
text("Combo Atual: " + comboAtual, (int)margemLateral, 70);
text("Maior Combo: " + maiorCombo, (int)margemLateral, 90);
textAlign(CENTER, CENTER);
if (aguardandoPausa && millis() - tempoVirada > tempoPausa) {
    if (cartas[cartaSelecionada1] == cartas[cartaSelecionada2]) {
        encontradas[cartaSelecionada1] = true;
        encontradas[cartaSelecionada2] = true;
        acertos++;
        comboAtual++;
    }
}
...

```



DESENVOLVIMENTO DE GAMES (5º PERÍODO)

Minigame 5: Jogo da Memória

Código do minigame

```
if (comboAtual > maiorCombo) maiorCombo = comboAtual;
    if (acertos == numPares) jogoFinalizado = true;
    } else {
        viradas[cartaSelecionada1] = false;
        viradas[cartaSelecionada2] = false;
        comboAtual = 0;
    }
    cartaSelecionada1 = -1;
    cartaSelecionada2 = -1;
    aguardandoPausa = false;
}
}

void mousePressed() {
    if (jogoFinalizado) {
        jogoFinalizado = false;
        jogoIniciado = false;
        inicializarJogo();
        return;
    }
    if (!jogoIniciado) {
        float yTema = height / 2 - 55;
        for (int i = 0; i < temas.length; i++) {
            if (mouseX > width / 2 - 100 && mouseX < width / 2 + 100 &&
                mouseY > yTema - 15 && mouseY < yTema + 15) {
                temaAtual = temas[i];
                carregarTema(temaAtual);
                return;
            }
            yTema += 30;
        }
        if (mouseX > botaoX && mouseX < botaoX + botaoLargura &&
            mouseY > botaoY && mouseY < botaoY + botaoAltura) {
            jogoIniciado = true;
        }
        return;
    }

    if (aguardandoPausa) return;
    int espacoHorizontal = 15;
    int espacoVertical = 15;
    int cartasPorLinha = 4;
    float larguraTotalGrid = (cartasPorLinha * larguraCarta) + ((cartasPorLinha - 1) *
    espacoHorizontal);
    float margemLateral = (width - larguraTotalGrid) / 2;
    float margemSuperior = 100;

    ...
}
```



DESENVOLVIMENTO DE GAMES (5º PERÍODO)

Minigame 5: Jogo da Memória

Código do minigame

```
for (int i = 0; i < numCartas; i++) {
    int coluna = i % cartasPorLinha;
    int linha = i / cartasPorLinha;
    int x = (int) (coluna * (larguraCarta + espacoHorizontal) + larguraCarta / 2 +
margemLateral);
    int y = (int) (linha * (alturaCarta + espacoVertical) + alturaCarta / 2 +
margemSuperior);

    if (!viradas[i] && !encontradas[i] &&
        mouseX > x - larguraCarta / 2 && mouseX < x + larguraCarta / 2 &&
        mouseY > y - alturaCarta / 2 && mouseY < y + alturaCarta / 2) {
        viradas[i] = true;
        if (cartaSelecionada1 == -1) {
            cartaSelecionada1 = i;
        } else if (cartaSelecionada2 == -1) {
            cartaSelecionada2 = i;
            tentativas++;
            tempoVirada = millis();
            aguardandoPausa = true;
        }
        break;
    }
}

void inicializarJogo() {
    cartas = new int[numCartas];
    viradas = new boolean[numCartas];
    encontradas = new boolean[numCartas];
    for (int i = 0; i < numPares; i++) {
        cartas[i * 2] = i;
        cartas[i * 2 + 1] = i;
    }
    embaralharCartas();
    tentativas = 0;
    acertos = 0;
    comboAtual = 0;
    maiorCombo = 0;
    cartaSelecionada1 = -1;
    cartaSelecionada2 = -1;
}

void embaralharCartas() {
    for (int i = cartas.length - 1; i > 0; i--) {
        int j = (int) random(i + 1);
        int temp = cartas[i];
        cartas[i] = cartas[j];
        cartas[j] = temp;
    }
}

...
```



DESENVOLVIMENTO DE GAMES (5º PERÍODO)

Minigame 5: Jogo da Memória

Código do minigame

```
imagens = new PImage[numPares];  
for (int i = 0; i < numPares; i++) {  
    imagens[i] = loadImage(linksTemas[indice][i]);  
}  
}
```

